



UNIVERSITÀ DEGLI STUDI DI SASSARI

INSTALLAZIONE E TEST DI DUE SIMULATORI ARCHITETTURALI: Gem5 e MarssX86

Studente:	Matricola:
Daniele Picciau	50056771
Monica Manai	50057077
Diego Perazzona	50057111

ANNO ACCADEMICO 2025-2026

1	Installazione e setup dei simulatori per Windows e macOS	3
1.1	Premessa fondamentale: architetture e compatibilità	3
1.2	Installazione e setup del simulatore Gem5	3
1.2.1	Installazione e setup di Gem5 su Windows	3
1.2.2	Installazione e setup di Gem5 su macOS	6
1.3	Installazione e setup del simulatore MarssX86	8
1.3.1	Installazione e setup di MarssX86 su Windows	8
1.3.2	Installazione e setup di MarssX86 su macOS	13
2	Esecuzione di un workload tramite i simulatori	16
2.1	Definizione di "piccolo workload"	16
2.2	Problema metodologico	17
2.2.1	Il caso Gem5 in modalità (SE)	17
2.2.2	Il caso MarssX86 (FS)	18
2.3	Strategia adottata	18
2.4	Metodologia e Configurazione degli Esperimenti	19
2.4.1	Matrice dei test e definizione delle Baseline	19
2.5	Esecuzione del workload per ARM e RISC-V (Gem5)	20
2.5.1	Installazione dei cross-compiler su Windows (x86_64)	20
2.5.2	Installazione dei cross-compiler su macOS (ARM)	21
2.5.3	Comandi per la build del workload	21
2.6	Esecuzione del workload per X86 (MarssX86)	22
2.6.1	Implementazione della metodologia ROI	23
2.6.2	Comandi per la build del workload	27
3	Analisi dei dati raccolti	29
3.1	Analisi delle tre configurazioni per l'architettura ARM	29
3.1.1	Statistiche generali su CPU e istruzioni	29
3.1.2	Statistiche Cache	30
3.1.3	Analisi dei Branch	30
3.1.4	Conclusioni	31
3.2	Analisi delle tre configurazioni per l'architettura RISC-V	32
3.2.1	Statistiche generali su CPU e istruzioni	32
3.2.2	Statistiche Cache	32
3.2.3	Analisi dei Branch	33
3.2.4	Conclusioni	33
3.3	Analisi delle tre configurazioni per l'architettura MarssX86	34
3.3.1	Statistiche generali su CPU e istruzioni	35
3.3.2	Statistiche Cache	35
3.3.3	Analisi dei Branch	36

3.3.4	Conclusioni	36
3.4	Analisi delle baseline tra le architetture	37
3.4.1	Statistiche generali su CPU e istruzioni	38
3.4.2	Statistiche Cache	38
3.4.3	Analisi dei Branch	38
3.4.4	Conclusioni	39

1 Installazione e setup dei simulatori per Windows e macOS

1.1 Premessa fondamentale: architetture e compatibilità

Prima di procedere, è essenziale capire che questi simulatori non sono normali applicazioni installabili con un semplice clic. Entrambi sono nati in ambito accademico per Linux, ma hanno esigenze molto diverse legate alla loro "età".

Questi due simulatori architetturali (Gem5 e MarssX86) nascono e vengono sviluppati nativamente per sistemi Unix-like.

- **macOS** è un sistema certificato Unix. "Parla la stessa lingua" di Linux, quindi può eseguire molti di questi programmi nativamente (direttamente dal terminale), con il solo aiuto di alcune librerie esterne.
- **Windows** ha un'architettura completamente diversa, per cui non può eseguire nativamente questi programmi. Per poterli eseguire è quindi necessario utilizzare delle soluzioni di **virtualizzazione o sottosistemi** per creare l'ambiente Unix necessario.

Bisogna anche considerare che, anche avendo un sistema Linux (o Unix), c'è il problema delle librerie software necessarie per compilare il codice (il "Time Gap").

- **Gem5** è un software **moderno e adattivo**, che viene continuamente sviluppato e aggiornato. Funziona senza problemi sui sistemi operativi moderni: su macOS lo si compila direttamente mentre su Windows si usa WSL con un'installazione Linux recente (Ubuntu 20.04/22.04).
- **MarssX86** è un progetto "legacy" (molto vecchio, fermo ad 2012 circa). Richiede compilatori (GCC vecchi) e librerie che sono state rimosse dai sistemi operativi moderni da anni. Nè macOS moderno né WSL possono eseguirlo facilmente, dunque è necessario creare una **Macchina virtuale** (con VirtualBox) che simuli un computer del 2014 con installato **Ubuntu 14.04**.

1.2 Installazione e setup del simulatore Gem5

Per quanto riguarda l'installazione del simulatore Gem5, la soluzione varia leggermente a seconda del sistema operativo utilizzato.

1.2.1 Installazione e setup di Gem5 su Windows

Per installare il simulatore su Windows la soluzione consigliata è **WSL** (Windows Subsystem for Linux). Questo sottosistema permette di avere un terminale Ubuntu vero e proprio integrato in Windows. Dunque, l'installazione si compone di diversi passaggi:

- **Passo 1: attivazione di WSL**
 1. Aprire la **PowerShell** come amministratore;
 2. Digitare `wsl --install`, premere invio e attendere il completamento del download;
 3. Al termine, seguire le istruzioni a schermo per creare *username* e *password* Ubuntu.

Una volta fatto ciò, la situazione iniziale nel terminale dovrebbe essere quella mostrata in figura 1, ed eseguendo il comando `pwd` sarà possibile vedere il proprio posizionamento all'interno del sottosistema WSL.

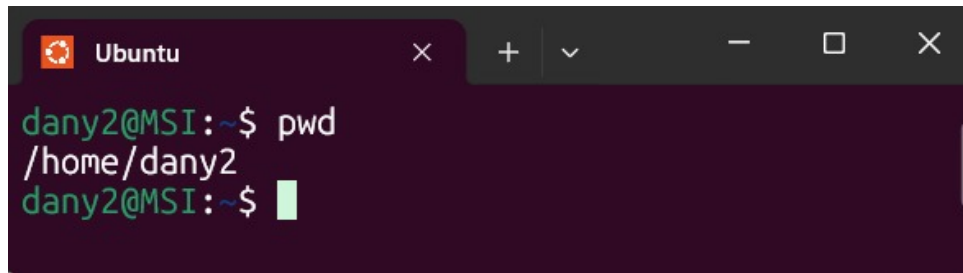


Figure 1: Schermata iniziale del sottosistema WSL

- **Passo 2: installazione delle dipendenze**

Nel terminale di Ubuntu appena aperto incollare questi comandi uno alla volta:

```
//Aggiorna i pacchetti  
sudo apt update && sudo apt upgrade -y
```

```
//Installa compilatori, Python e librerie necessarie a Gem5  
sudo apt install build-essential git m4 scons zlib1g zlib1g-dev  
libprotobuf-dev protobuf-compiler libprotoc-dev libgoogle-perftools-  
dev python3-dev python-is-python3 libboost-all-dev pkg-config libpng  
-dev libhdf5-dev libcapstone-dev -y
```

Note: facendo copia-incolla dei comandi potrebbero verificarsi dei problemi dovuti ai "-" letti dal terminale di Ubuntu come caratteri speciali, in tal caso è consigliato riscriverli a mano.

- **Passo 3: scaricare e compilare Gem5**

Prima di **scaricare** Gem5 è consigliato creare una cartella adatta a poterlo contenere. Quest'operazione viene svolta principalmente perché in futuro dovremo scaricare anche il simulatore architetturale **MarssX86**: così facendo avremo entrambi i simulatori in una cartella apposita.

Senza spostarsi dalla posizione iniziale /home/NomeUtente si va a creare la cartella:

```
mkdir dev_local // Crea la cartella dev_local  
cd dev_local // Ci si sposta dentro dev_local per scaricare gem5
```

Una volta posizionati in /home/utente/dev_local è possibile utilizzare il comando `git clone` per scaricare il simulatore gem5 dal repository ufficiale

```
//Clonare il repository del progetto nel proprio workspace  
git clone https://github.com/gem5/gem5.git
```

Quando la clonazione sarà completata, all'interno di dev_local sarà presente un'altra cartella nominata "gem5" al cui interno saranno presenti tutti i file necessari per la compilazione del simulatore.

Prima di far partire la compilazione è necessario fare alcune **premesse importanti**.

Utilizzare un calcolatore con 16GB o meno di memoria RAM non basterà: Gem5 è enorme e usa template C++ complessi, ogni core che compila può occupare anche 2-3 GB di RAM. Inoltre, considerando che Windows utilizza parte di questa RAM, la situazione si complica ulteriormente.

Una possibile soluzione a questo problema, nel caso in cui non si disponesse di un calcolatore

con la quantità di RAM necessaria, è quella di "truccare" WSL per dargli più memoria usando il disco fisso come se fosse RAM (operazione di Swap della memoria):

1. Aprire il blocco note di windows;
2. Incollare esattamente queste righe:

```
[wsl2]
memory=10GB
swap=16GB
```

In questo modo stiamo dando a WSL massimo 10GB di RAM vera, ma aggiungiamo 16GB di "memoria lenta" su disco per evitare che la compilazione vada in crash.

3. Salvare il file con nome `.wslconfig` (assicurandosi che non ci sia `.txt` alla fine!) nella propria cartella utente principale: `C:\Utenti\NomeUtente\.wslconfig`
4. Aprire la PowerShell (sempre in modalità amministratore) e spegnere WSL per applicare la modifica con il comando `wsl --shutdown`

Una volta fatte queste modifiche, è finalmente possibile far partire la compilazione: riaprendo il terminale Ubuntu e posizionandosi in `/home/NomeUtente/dev_local/gem5` deve essere scritto il seguente comando:

```
scons build/ARM/gem5.opt -j 1
```

Tramite di esso la compilazione viene fatta partire utilizzando un solo core della cpu. Nonostante l'utilizzo dello "Swap" con la memoria del disco, la compilazione può essere talmente pesante che utilizzando più di un core per la compilazione si rischierebbe di saturare la RAM mandando in crash la compilazione. È **assolutamente "normale"** che i tempi di compilazione varino da 1h a 2h in base alla cpu che si ha a disposizione.

Note: non appena la compilazione del simulatore ARM termina, per incominciare la compilazione del simulatore RISC-V basterà reinviare il comando `scons` qui sopra, sostituendo ARM con RISC-V.

• **Passo 4: test di funzionamento**

Per testare che il simulatore sia stato installato correttamente è possibile lanciare un comando di prova già incluso al suo interno.

Sempre posizionati in `/home/NomeUtente/dev_local/gem5`, lanciare questo comando:

```
./build/ARM/gem5.opt configs/deprecated/example/se.py --cmd=tests/test-progs/hello/bin/arm/linux/hello
```

```
dany2@MSI:~/dev_local/gem5$ ./build/ARM/gem5.opt configs/deprecated/example/se.py --cmd=tests/test-progs/hello/bin/arm/linux/hello
gem5 Simulator System.  https://www.gem5.org
gem5 is copyrighted software; use the --copyright option for details.

gem5 version 25.1.0.0
gem5 compiled Jan 14 2026 01:11:53
gem5 started Jan 14 2026 01:41:36
gem5 executing on MSI, pid 576
command line: ./build/ARM/gem5.opt configs/deprecated/example/se.py --cmd=tests/test-progs/hello/bin/arm/linux/hello

warn: The se.py script is deprecated. It will be removed in future releases of gem5.
Global frequency set at 1000000000 ticks per second
warn: No dot file generated. Please install pydot to generate the dot file and pdf.
src/mem/dram_interface.cc:690: warn: DRAM device capacity (8192 Mbytes) does not match the address range assigned (512 Mbytes)
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat is a stat that does not belong to any statistics::
Group. Legacy stat is deprecated.
system.remote_gdb: Listening for connections on port 7000
**** REAL SIMULATION ****
Hello world!
Exiting @ tick 2920000 because exiting with last active thread context
Simulated exit code not 0! Exit code is 13
```

Figure 2: test di simulazione per Gem5

L'esecuzione di questo comando comporterà la creazione, all'interno della cartella "m5out" situata all'interno della cartella "gem5", di **due file fondamentali**:

- **"stats.txt"**: Contiene tutte le statistiche (cicli, cache miss, istruzioni eseguite, ecc.). È il file di testo enorme che si dovrà analizzare dopo l'esecuzione vera e propria del workload;
- **"config.ini"**: Contiene la descrizione dettagliata dell'hardware appena simulato (utile per verificare se è stata davvero usata la CPU che si voleva).

1.2.2 Installazione e setup di Gem5 su macOS

Sebbene sia teoricamente possibile compilare nativamente su macOS, la gestione delle dipendenze (in particolare Python e Protobuf) risulta spesso instabile a causa degli aggiornamenti di sistema Apple.

Per garantire la **massima riproducibilità** e stabilità, la soluzione raccomandata per macOS è l'utilizzo di un container **Docker** basato su Ubuntu 22.04. Come per WSL, questo approccio permette di avere un ambiente di sviluppo basato su Linux nativo.

- **Passo 1: preparazione del container**

1. Scaricare e installare Docker desktop dal sito ufficiale <https://www.docker.com/>;
2. È consigliato aprire il terminale posizionandosi nella cartella del proprio apple account come mostrato in figura 3;

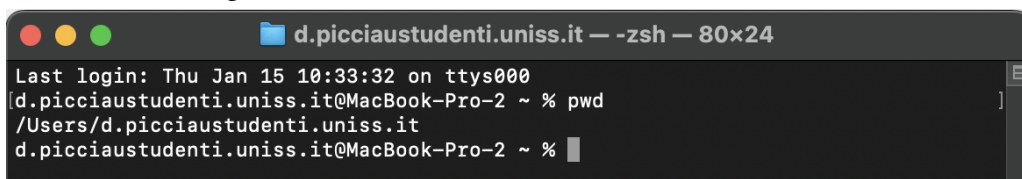


Figure 3: posizionamento iniziale per l'installazione di docker

3. Creare il proprio workspace che conterrà il simulatore Gem5 e in futuro MarssX86.

```
mkdir dev_local
cd dev_local
```

- **Passo 2: installazione delle dipendenze**

1. All'interno di /Users/NomeUtente/dev_local, installare tutte le dipendenze necessarie per la creazione del container di Ubuntu 22.04:

```
docker pull ghcr.io/gem5/ubuntu-24.04_all-dependencies:v25-1
```

2. Una volta che il workspace è stato creato è possibile far partire il container di Ubuntu (sempre stando posizionati in /Users/NomeUtente/dev_local):

```
docker run --volume /Users/NomeUtente/dev_local/gem5:/gem5 --rm -it
ghcr.io/gem5/ubuntu-24.04_all-dependencies:v24-0
```

Il completamento di questo comando dovrebbe aprire una riga di comando di questo tipo, ad indicare che siamo all'interno del container Ubuntu.

```
root@{...}:/#
```

- **Passo 3: scaricare e compilare Gem5**

All'interno del container Ubuntu appena creato, è possibile **scaricare** il repository ufficiale tramite il comando `git clone`:

```
root@{...}:/# git clone https://github.com/gem5/gem5.git
```

Verrà creata una cartella "gem5" al cui interno saranno presenti tutti i file necessari per la compilazione del simulatore.

Come specificato al "**Passo 3**" del capitolo 1.2.1 la compilazione richiede una grande quantità di RAM per cui è richiesto un calcolatore con 16 o più GB di memoria RAM (solo per la compilazione di Gem5!). Dopo alcuni test da noi effettuati è risultato che la compilazione su un calcolatore con 8GB di RAM potrebbe impiegare anche mezza giornata!

Proprio per questo motivo, anche disponendo di 16GB di memoria RAM è necessario andare nella GUI di Docker impostazioni → resources → advanced, e da qui cambiare le seguenti impostazioni:

- **Memory limit:** impostiamo la capacità massima di memoria RAM che Docker può utilizzare. È consigliato impostarlo quasi al massimo, (ad esempio, con 16 GB di RAM a disposizione impostarlo a 14/15) per permettere al sistema operativo (macOS) di "sopravvivere"
- **Swap:** Invece, tramite questa impostazione, indichiamo quanto spazio di "memoria lenta" prendere dal disco nel caso in cui la RAM vada in saturazione a causa della pesantezza di compilazione.

Una volta che il repository è stato clonato ci si sposta all'interno della cartella "gem5" per far partire la **compilazione del simulatore**, utilizzando 1 o al massimo 2 core per evitare che la compilazione vada in crash a causa della saturazione della RAM a disposizione.

```
// Mi sposto all'interno della cartella gem5
root@{...}:/# cd gem5

// Compilazione ARM
root@{...}:/gem5# scons build/ARM/gem5.opt -j1
```

Come per Windows, anche la compilazione su macOS può impiegare da 1h a 2h, ed è "**normale**" vista la mole di file da scaricare.

Note: non appena la compilazione del simulatore ARM termina, per incominciare la compilazione del simulatore RISC-V basterà reinviare il comando scons qui sopra, sostituendo ARM con RISC-V.

Passo 4: test di funzionamento

Per testare che il simulatore sia stato installato correttamente è possibile lanciare un comando di prova già incluso al suo interno.

Sempre posizionati in root@1ab1ef08f2f:/gem5#, lanciare questo comando:

```
./build/ARM/gem5.opt configs/deprecated/example/se.py --cmd=tests/test-
progs/hello/bin/arm/linux/hello
```

L'esecuzione di questo comando comporterà la creazione, all'interno della cartella "m5out" situata all'interno della cartella "gem5", di **due file fondamentali**:

- **"stats.txt"**: Contiene tutte le statistiche (cicli, cache miss, istruzioni eseguite, ecc.). È il file di testo enorme che si dovrà analizzare dopo l'esecuzione vera e propria del workload;
- **"config.ini"**: Contiene la descrizione dettagliata dell'hardware appena simulato (utile per verificare se è stata davvero usata la CPU che si voleva).

1.3 Installazione e setup del simulatore MarssX86

Nonostante sia un processo abbastanza lungo e tedioso la scelta consigliata per effettuare l'installazione del simulatore, sia per Windows che per macOS, è utilizzare Docker.

1.3.1 Installazione e setup di MarssX86 su Windows

L'utilizzo di Docker è la scelta raccomandata per mantenere un flusso di lavoro unificato. Grazie all'integrazione con WSL 2, è possibile avviare l'ambiente "legacy" necessario per MarssX86 direttamente dallo stesso terminale utilizzato per Gem5 (come visto al passo 1 del capitolo 1.2.1), senza dover configurare macchine virtuali esterne pesanti.

- **Passo 1: Configurazione (WSL + Docker)**

1. Scaricare e installare Docker desktop dal sito ufficiale <https://www.docker.com/>;
2. Aprire docker desktop su Windows;
3. Andare nelle impostazioni (icona dell'ingranaggio in alto a destra);
4. Spostarsi nella sezione **resources** e poi in **WSL integration**;
5. Assicursi di attivare la casella "Ubuntu" per abilitare l'integrazione;
6. Cliccare su "apply e restart"

A questo punto, aprendo il terminale WSL, è possibile verificarne il funzionamento tramite il comando `docker -version`.

- **Passo 2: preparazione del container**

1. Nel terminale WSL spostarsi all'interno della cartella `/home/NomeUtente/dev_local` (come fatto al passo 3 del capitolo 1.2.1), e creare una cartella per il progetto Marss con al suo interno il file di definizione dell'ambiente:

```
mkdir marss
cd marss
nano Dockerfile
```

All'interno dell'editor che si aprirà dopo l'ultimo comando (`nano Dockerfile`), incollare il seguente contenuto che definisce un ambiente Ubuntu 18.04 con tutte le dipendenze necessarie.

Nonostante MarssX86 sia stato scritto in un ambiente Ubuntu 14.04 (risalente al 2012), non è possibile replicare tale ambiente, poiché da molti anni i server hanno cancellato tutti i file. Dunque una possibile strategia è quella di usare un sistema operativo più recente (Ubuntu 18.04) i cui server funzionano perfettamente, ma installare manualmente i "vecchi" compilatori.

```
# Usiamo Ubuntu 18.04 che ha server ancora attivi e stabili
FROM ubuntu:18.04

# Aggiorniamo i repository (funzionerà senza errori 404)
RUN apt-get update

# Installiamo i tool di base e specificamente GCC 4.8 / G++ 4.8
# MarssX86 richiede queste versioni antiche per compilare
RUN apt-get install -y build-essential git scons zlib1g-dev python2.7
python-minimal g++ \
vim libstdc++11-dev gcc-4.8 g++-4.8
```

```

# Configuriamo il sistema per usare GCC 4.8 come default assoluto
# Questo "inganna" Marss facendogli credere di essere su un vecchio
sistema
RUN update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-4.8
    100 && \
    update-alternatives --install /usr/bin/g++ g++ /usr/bin/g++-4.8
        100

# Cartella di lavoro
WORKDIR /root/marss

CMD ["/bin/bash"]

```

2. Rimanendo posizionati in `/home/NomeUtente/dev_local/marss`, lanciare il comando di build per l'ambiente Ubuntu. Verrà richiesta una password, inserire quella dell'account Ubuntu come fatto al **"Passo 1"** del capitolo 1.2.1.

```
sudo docker build --no-cache -t marss_env .
```

3. Infine, è possibile creare l'ambiente Ubuntu scelto. Servirà la password.

```
sudo docker run -it --name marss_container -v "$PWD":/root/marss
    marss_env
```

Nota importante: l'aggiunta del comando `-v "$PWD":/root/marss` crea un collegamento diretto tra la cartella in cui ci si trova (`/home/NomeUtente/dev_local/marss`) nel disco e la cartella di lavoro "virtuale" del container. Senza di esso non si vedrebbero le modifiche effettuate nel container, costringendo a copiare ogni file che si vuole utilizzare nel disco del computer, tramite comandi aggiuntivi. Ovviamente, il container rimane necessario per compilare/eseguire i file dell'ambiente Ubuntu scelto.

Una volta fatto ciò, si dovrebbe essere riusciti ad entrare nell'ambiente di simulazione, e a schermo dovrebbe vedersi questo:

```
root@{...}:~/marss#
```

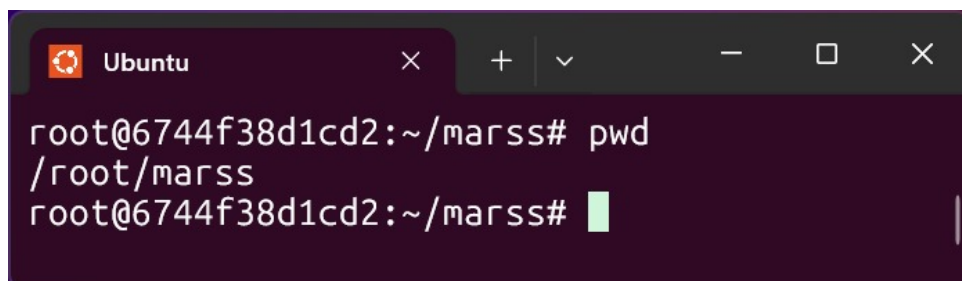


Figure 4: ambiente di simulazione del MarssX86

Inserendo subito il comando `"pwd"` si potrà vedere come ci si trova in `/root/marss` come specificato nel Dockerfile.

4. Per **tornare alla bash di Ubuntu** basterà scrivere il comando `"exit"` da qualsiasi posizione, mentre per rientrare nell'ambiente già creato servirà un comando differente da lanciare (posizionati in `/home/NomeUtente/dev_local/marss`):

```
docker start -ai marss_container
```

Invece, per rimuovere il container serve prima sapere il container id. Nell'ambiente Ubuntu (quindi dopo avere fatto "exit") utilizzare il seguente comando:

```
docker ps
```

Subito dopo

```
docker rm <container id>
```

- **Passo 3: scaricare e compilare MarssX86**

Dopo essere riusciti ad entrare dentro l'ambiente di simulazione, bisogna **scaricare il repository** del progetto ufficiale (posizionati in /root/marss):

```
root@{...}:~/marss# git clone https://github.com/avadhpatel/marss.git
```

Una volta completata la clonazione, all'interno di /root/marss verrà scaricata un'ulteriore cartella "marss" contenente i file relativi al repository.

Anche in questo caso la compilazione risulta abbastanza pesante a causa della quantità di RAM richiesta. Fortunatamente, Docker desktop su Windows eredita da WSL il file .wslconfig (**passo 3** del capitolo 1.2.1) creato per fare in modo che la compilazione non vada in crash.

Prima di **procedere con la compilazione** ci sono alcune problematiche che vanno risolte, relative alle discrepanze tra l'utilizzo di comandi moderni e compilatori vecchi oppure causate da librerie di sicurezza moderne (già presenti nella versione 18.04 di Ubuntu che stiamo utilizzando) che bloccano la compilazione del file finale di compilazione. Queste problematiche sono sorte in fase di compilazione stessa, e avendole individuate, è necessario risolverle per garantire il funzionamento della compilazione.

1. Posizionarsi in /root/marss/marss/ptlsim e aprire il file "SConstruct" che rappresenta il "cuore" del simulatore PTLsim:

```
root@{...}:~/marss/marss/ptlsim# apt-get install -y nano
root@{...}:~/marss/marss/ptlsim# nano SConstruct
```

2. Una volta all'interno dell'editor di testo nano per il file "SConstruct" cercare la riga:

```
env.Append(CCFLAGS = ['-fdiagnostics-color=always'])
```

ed eliminarla o commentarla utilizzando un "#". Per aiutarsi è possibile entrare in modalità ricerca con la combinazione di comandi ctrl + w.

Subito sotto questa riga, ne dovrebbe essere presente un'altra di questo tipo:

```
env.Append(CCFLAGS = ' -std=gnu++11 ')
```

la quale deve essere modificata in questo modo:

```
env.Append(CCFLAGS = ' -std=gnu++11 -U_FORTIFY_SOURCE ')
```

In questo modo stiamo dicendo al compilatore di "disabilitare i controlli di sicurezza moderni per il codice vecchio del repository".

Premere ctrl+o per salvare le modifiche, invio, e ctrl+x per uscire dall'editor di testo.

3. Tornati nell'ambiente di simulazione bisogna effettuare lo stesso procedimento nel file "SConstruct" principale, quindi non quello presente in /root/marss/marss/ptlsim ma in /root/marss/marss/, quindi:

```
root@{...}:~/marss/marss# nano SConstruct
```

Una volta all'interno dell'editor di testo, bisogna scendere nel file fino a trovare una istruzione condizionale di questo tipo:

```
if int(pretty_printing) :
    base_env = Environment(
        CXXCOMSTR = compile_source_message,
        CREATECOMSTR = create_header_message,
        CCCCOMSTR = compile_source_message,
        SHCCCOMSTR = compile_shared_source_message,
        SHCXXCOMSTR = compile_shared_source_message,
        ARCOMSTR = link_library_message,
        RANLIBCOMSTR = ranlib_library_message,
        SHLINKCOMSTR = link_shared_library_message,
        LINKCOMSTR = link_program_message,
    )
else:
    base_env = Environment()
```

Questo blocco serve a creare l'ambiente di compilazione (base_env). Bisogna intercettare quella variabile appena è stata creata e aggiungere il "flag salvavita" che disabilita i controlli di sicurezza troppo moderni. Dopo l'else bisogna aggiungere le seguenti righe:

```
...
...

else:
    base_env = Environment()

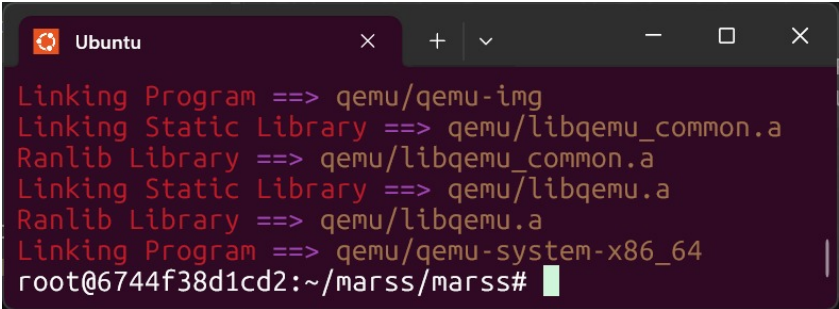
base_env.Append(CFLAGS = ['-U_FORTIFY_SOURCE'])
base_env.Append(CCFLAGS = ['-U_FORTIFY_SOURCE'])
```

Fatto ciò, nuovamente ctrl+o per salvare le modifiche, invio, e poi ctrl+x per tornare nell'ambiente di simulazione.

Effettuate queste modifiche è finalmente possibile **compilare** tramite il comando:

```
root@{...}:~/marss/marss# scons -Q C=1 debug=0
```

Come risultato finale si dovrebbe visualizzare un comando di questo tipo:



```
Linking Program ==> qemu/qemu-img
Linking Static Library ==> qemu/libqemu_common.a
Ranlib Library ==> qemu/libqemu_common.a
Linking Static Library ==> qemu/libqemu.a
Ranlib Library ==> qemu/libqemu.a
Linking Program ==> qemu/qemu-system-x86_64
root@6744f38d1cd2:~/marss/marss#
```

Al termine della compilazione è possibile verificare la presenza dell'eseguibile tramite il seguente comando:

```
root@{...}:~/marss/marss# ./qemu/qemu-system-x86_64 --version
```

Se in risposta si riceve un qualcosa del tipo:

```
QEMU emulator version 0.14.1, Copyright (c) 2003-2008 Fabrice Bellard
```

allora il simulatore è pronto all'uso.

- **Passo 4: installazione immagine disco rigido**

Dopo aver completato la fase di installazione del simulatore con l'ambiente Ubuntu, si passa alla fase di installazione del sistema operativo vero e proprio.

A differenza di Gem5 che carica direttamente l'eseguibile, MarssX86 è un simulatore Full System (FS): simula un PC intero (BIOS, scheda madre, disco rigido). Quindi, per farlo funzionare, ci serve un'immagine del disco rigido con dentro un Sistema Operativo (Ubuntu vecchio).

1. Si rende necessario quindi, preparare la cartella dei dischi. Posizionati in root/marss/ per creare una cartella "disks":

```
mkdir disks  
cd disks
```

2. Una volta creata la cartella, spostarsi in root/marss/marss/disks e installare il comando "wget" per poter poi installare il sistema operativo da emulare tramite l'ambiente Ubuntu preparato precedentemente.

```
root@{...}:~/marss/marss/disks# apt-get install -y wget bzip2
```

Installerà tutte le librerie necessarie al suo funzionamento, ci sarà da aspettare qualche secondo.

3. A questo punto è tutto pronto per installare il sistema operativo.

Il sistema operativo scelto è ottenibile al seguente link https://people.debian.org/~aurel32/qemu/amd64/debian_squeeze_amd64_standard.qcow2.

Si tratta di *Debian Squeeze*, una distribuzione di Linux rilasciata nel febbraio del 2011.

La scelta di questa distribuzione specifica è dettata da vincoli di compatibilità:

- **Sincronizzazione Temporale:** MarssX86 è stato sviluppato intorno al 2011-2012, basandosi su una versione di QEMU (0.14) di quel periodo. I simulatori Full System simulano l'hardware di quell'epoca, e Debian Squeeze, essendo del 2011, "parla la stessa lingua" del simulatore.
- **Leggerezza (no graphic):** l'immagine scaricata è una versione "Standard/Server", priva di interfaccia grafica (niente finestre, niente mouse), e questo è cruciale poiché riduce il consumo di RAM e CPU simulata

Dunque, per installare la distribuzione scelta si utilizza il seguente comando:

```
root@{...}:~/marss/marss/disks# wget https://people.debian.org/~  
aurel32/qemu/amd64/debian_squeeze_amd64_standard.qcow2
```

4. Nonostante l'immagine del disco sia idonea per MarssX86, è necessaria una versione precedente. L'immagine scaricata è in formato QCOW2 versione 3 (introdotto da QEMU 1.1 nel 2012), ma MarssX86 usa QEMU 0.14 (2011), che conosce solo la versione 2.

Per risolvere questo problema legato alla versione, è necessario effettuare un "downgrade" dell'header del file del disco virtuale per renderlo leggibile dal vecchio simulatore:

```
root@{...}:~/marss/marss/disks# apt-get update && apt-get install -y  
qemu-utils
```

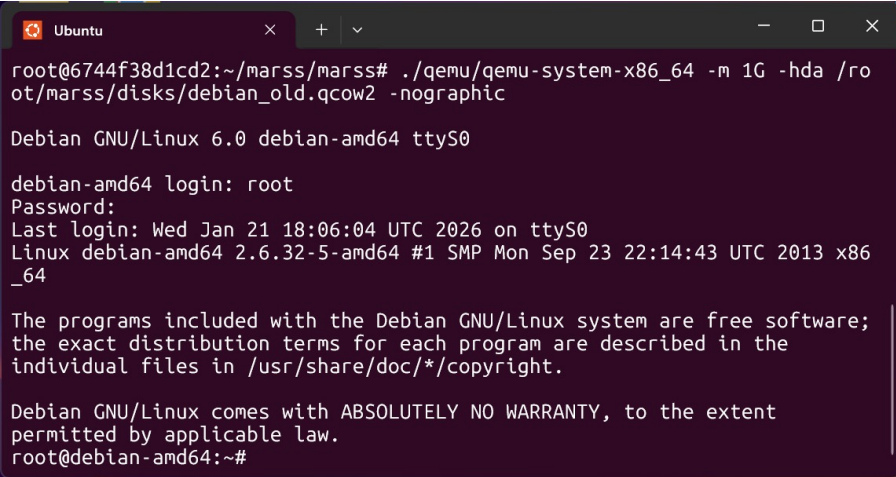
```
root@{...}:~/marss/marss/disks# qemu-img convert -f qcow2 -O qcow2 -o  
compat=0.10 debian_squeeze_amd64_standard.qcow2 debian_old.qcow2
```

`compat=0.10` forza il formato a essere leggibile dai simulatori del 2011, creando una nuova immagine del disco *debian_old.qcow2*.

5. Ora, per far partire a distribuzione Debian, basterà lanciare il seguente comando, posizionati in `/root/marss/marss`:

```
root@{...}:~/marss/marss# ./qemu/qemu-system-x86_64 -m 1G -hda /root/  
marss/disks/debian_old.qcow2 -nographic
```

Verrà richiesto login e password, le credenziali saranno "root" per entrambi i campi.



```
Ubuntu  
root@6744f38d1cd2:~/marss/marss# ./qemu/qemu-system-x86_64 -m 1G -hda /ro  
ot/marss/disks/debian_old.qcow2 -nographic  
  
Debian GNU/Linux 6.0 debian-amd64 ttyS0  
  
debian-amd64 login: root  
Password:  
Last login: Wed Jan 21 18:06:04 UTC 2026 on ttyS0  
Linux debian-amd64 2.6.32-5-amd64 #1 SMP Mon Sep 23 22:14:43 UTC 2013 x86  
_64  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
root@debian-amd64:~#
```

Ruolo nell'architettura della simulazione

La procedura si presenta abbastanza complessa, dunque, per chiarire il funzionamento complessivo, facciamo un recap:

1. **Host Fisico:** il PC Windows (server WSL) o il PC macOS.
2. **Container Docker:** l'ambiente che contiene le librerie necessarie per compilare ed eseguire il simulatore.
3. **Simulatore (MarssX86/QEMU):** il programma che finge di essere un computer fisico (crea CPU, RAM e Disco finti).
4. **Guest OS (Debian Squeeze):** Il sistema operativo che "vive" dentro il disco finto e che esegue effettivamente il codice C.

1.3.2 Installazione e setup di MarssX86 su macOS

Dal momento che, per simulare MarssX86 su Windows si sta utilizzando Docker, gran parte dei passaggi da effettuare su macOS rimangono identici.

Tuttavia, per impostazione predefinita, Docker su un dispositivo che monta un processore apple silicon crea un contenitore ARM64 (aarch64). MarssX86 è un simulatore più vecchio

progettato esclusivamente per architetture x86_64 (Intel/AMD), dunque ne consegue che se si andasse ad effettuare il seguente script di build (SConstruct):

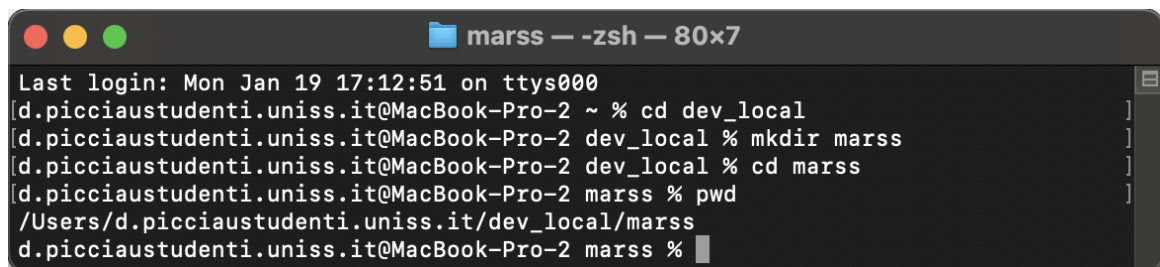
```
root@{...}:~/marss/marss# scons -Q C=1 debug=0
```

verrebbe rilevato che la CPU è basata su ARM e il processo verrebbe interrotto non sapendo come effettuare la compilazione.

A questo punto, facendo riferimento al capitolo 1.3.1 è necessario eliminare il "**passo 1**" poiché WSL non viene utilizzato, revisionare il "**passo 2**", modificando semplicemente i comandi di build e creazione dell'ambiente Ubuntu, e mantenere il "**passo 3**" invariato.

1. Come spiegato al "**passo 1**" del capitolo 1.2.2 scaricare Docker dal link fornito. Inoltre, è consigliato creare una cartella "marss" all'interno di "dev_local", per mantenere separati i flussi di lavoro tra i due simulatori.

Per fare ciò è necessario essere posizionati in /Users/NomeUtente/dev_local/marss



```
Last login: Mon Jan 19 17:12:51 on ttys000
[d.picciaustudenti.uniss.it@MacBook-Pro-2 ~ % cd dev_local
[d.picciaustudenti.uniss.it@MacBook-Pro-2 dev_local % mkdir marss
[d.picciaustudenti.uniss.it@MacBook-Pro-2 dev_local % cd marss
[d.picciaustudenti.uniss.it@MacBook-Pro-2 marss % pwd
/Users/d.picciaustudenti.uniss.it/dev_local/marss
[d.picciaustudenti.uniss.it@MacBook-Pro-2 marss %
```

2. All'interno della cartella "marss", come fatto per Windows, bisogna creare il "Dockerfile" all'interno del quale definire l'ambiente Ubuntu 18.04, dunque:

```
nano Dockerfile
```

e in seguito inserire le seguenti righe di codice:

```
# Usiamo Ubuntu 18.04 che ha server ancora attivi e stabili
FROM ubuntu:18.04

# Aggiorniamo i repository (funziona senza errori 404)
RUN apt-get update

# Installiamo i tool di base e specificamente GCC 4.8 / G++ 4.8
# MarssX86 richiede queste versioni antiche per compilare
RUN apt-get install -y build-essential git scons zlib1g-dev python2.7
python-minimal g++ \
    vim libstdc++12-dev gcc-4.8 g++-4.8

# Configuriamo il sistema per usare GCC 4.8 come default assoluto
# Questo "inganna" Marss facendogli credere di essere su un vecchio
sistema
RUN update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-4.8 100
&& \
    update-alternatives --install /usr/bin/g++ g++ /usr/bin/g++-4.8 100

# Cartella di lavoro
WORKDIR /root/marss
```

```
CMD ["/bin/bash"]
```

3. Rimanendo sempre posizionati in `/Users/NomeUtente/dev_local/marss` lanciare il comando di build per l'ambiente Ubuntu.

```
docker build --platform linux/amd64 --no-cache -t marss_env .
```

Questo dice a Docker di fingere che sia una macchina Intel durante la costruzione.

Note: questo passaggio richiederà più tempo rispetto a prima a causa dell'emulazione.

4. Subito dopo creare e far partire il container, specificando la piattaforma:

```
docker run --platform linux/amd64 -it --name marss_container -v "$PWD  
:/root/marss marss_env
```

Una volta lanciato questo comando si dovrebbe essere riusciti ad entrare nel container e a terminale si dovrebbe visualizzare questo:

```
root@{...}:~/marss#
```

5. Dopo essere riusciti ad entrare all'interno del container, i seguenti passaggi sono tali e quali a quelli descritti dal "**passo 3**" in poi del capitolo 1.3.1.

2 Esecuzione di un workload tramite i simulatori

L'obiettivo di questo progetto è analizzare e confrontare le prestazioni di tre diverse architetture (ISA): ARM, RISC-V e x86, utilizzando un workload computazionale standardizzato. Per poter eseguire questo confronto fra le differenti architetture è imperativo stabilire una metodologia che ne garantisca l'equità.

2.1 Definizione di "piccolo workload"

Le istruzioni del progetto richiedono l'esecuzione di un "piccolo workload" con tracing e statistiche. Per garantire la riproducibilità e la confrontabilità tra ARM, RISC-V e x86, il workload non deve dipendere da librerie di sistema complesse che potrebbero variare tra le architetture.

Il candidato ideale per questo workload è un algoritmo di moltiplicazione di matrici (Matrix Multiplication) denso. Questo kernel computazionale offre diversi vantaggi analitici:

- **Prevedibilità:** l'accesso alla memoria è regolare, permettendo di analizzare l'efficacia delle gerarchie di cache.
- **Intensità Computazionale:** stressa le unità funzionali della CPU e la logica di pipelining, evidenziando le differenze tra l'approccio RISC (molte istruzioni semplici) e CISC (istruzioni complesse con accessi in memoria impliciti).
- **Portabilità:** può essere scritto in C standard (ANSI C) e compilato staticamente per tutte e tre le architetture senza modifiche al codice sorgente.

Il codice sorgente C proposto per l'esperimento dovrà essere compilato in tre binari distinti utilizzando cross-compiler specifici per ARM, RISC-V e x86, come dettagliato nelle sezioni successive, ed è il seguente:

```
#include <stdio.h>
#include <stdlib.h>

#define N 64 // Dimensione ridotta per simulazione rapida

void matrix_multiply(double *A, double *B, double *C, int size) {
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            double sum = 0.0;
            for (int k = 0; k < size; k++) {
                sum += A[i * size + k] * B[k * size + j];
            }
            C[i * size + j] = sum;
        }
    }
}

int main() {
    size_t bytes = N * N * sizeof(double);
    double *A = (double *)malloc(bytes);
    double *B = (double *)malloc(bytes);
```

```

double *C = (double *)malloc(bytes);

// Inizializzazione deterministica
for (int i = 0; i < N * N; i++) {
    A[i] = 1.0;
    B[i] = 2.0;
}

printf("Inizio Benchmark Moltiplicazione Matrici %dx%d\n", N, N);
matrix_multiply(A, B, C, N);
printf("Fine Benchmark\n");

return 0;
}

```

2.2 Problema metodologico

Una delle sfide principali di questo confronto risiede nella natura diversa degli ambienti di simulazione disponibili:

- **Gem5** offre la modalità **Systemcall Emulation (SE)** la quale non simula un vero computer con un disco rigido e un sistema operativo completo. Permette di simulare l'esecuzione di binari user-space evitando di modellare i dispositivi o un OS, emulando la maggior parte dei servizi a livello di sistema;
- **MarssX86** opererà in modalità **Full System (FS)**, simulando un'intera macchina fisica, incluso il sistema operativo guest, i device e gli interrupt.

2.2.1 Il caso Gem5 in modalità (SE)

Nella compilazione standard (dinamica), l'eseguibile generato è incompleto: esso contiene riferimenti a funzioni esterne (come printf o malloc presenti nella glibc) ma non il loro codice macchina. Al momento dell'avvio, prima ancora che venga eseguita la funzione main(), il sistema operativo invoca il Dynamic Linker, il quale deve:

- Esplorare il filesystem alla ricerca delle librerie condivise (.so) richieste;
- Caricarle nella memoria virtuale del processo;
- Risolvere i simboli, collegando le chiamate del programma agli indirizzi di memoria delle librerie appena caricate;

Proprio per questo motivo, questo **meccanismo risulta critico** in modalità SE, poiché:

- **Assenza di Filesystem Virtuale Completo:** Gem5 in modalità SE non dispone di un filesystem guest nativo. Quando il Dynamic Linker (che è un programma a sé stante) cerca le librerie in percorsi standard (es. /lib/ o /usr/lib/), tali percorsi potrebbero non esistere nell'ambiente simulato o non coincidere con i percorsi delle librerie cross-compileate presenti sulla macchina host.
- **Complessità delle Syscall:** Il Dynamic Linker fa un uso intensivo di chiamate di sistema per mappare file in memoria (open, mmap). Sebbene Gem5 emuli le syscall comuni (come read()), la gestione complessa del caricamento dinamico può portare a errori di path lookup o incompatibilità di ABI (Application Binary Interface) tra host e guest.

Per ovviare a queste problematiche e garantire la riproducibilità degli esperimenti si rende dunque necessario imporre dei vincoli specifici sulla fase di compilazione e linking del workload che viene compilato staticamente utilizzando il flag `-static`.

Con il linking statico, tutte le dipendenze necessarie (inclusa la `libc`) vengono incorporate fisicamente all'interno del file binario al momento della compilazione. Ciò rende l'eseguibile autosufficiente:

- Elimina la necessità del Dynamic Linker in fase di avvio;
- Rimuove la dipendenza da librerie esterne durante la simulazione;
- Permette a Gem5 di caricare il binario in memoria ed eseguire direttamente il codice utente, intercettando ed emulando puntualmente le sole System Call esplicite generate dal programma (es. output su terminale), garantendo stabilità e portabilità tra diversi ambienti host.

2.2.2 Il caso MarssX86 (FS)

Nonostante il simulatore MarssX86 sia adatto a far girare un vero sistema operativo, con disco virtuale, un kernel Linux vero e delle librerie installate dentro l'immagine del disco (quindi in modalità Full System), rimane un problema di **compatibilità delle versioni**, dovuto principalmente al fatto che si tratta di un simulatore datato.

L'ambiente docker che viene utilizzato fornisce Ubuntu 18.04, uscito nel 2018, e utilizza la libreria `glibc` versione 2.27. Invece, le immagini del disco che si devono utilizzare per il corretto funzionamento su MarssX86 sono solitamente basate su ubuntu 10.04 o 12.04 (relative agli anni 2010 – 2012) e utilizzano una libreria `glibc` versione 2.15 o inferiore.

Purtroppo, come specificato al "**passo 2**" del capitolo 1.3.1, tali versioni dell'ambiente ubuntu sono state eliminate dai server.

La libreria standard di C non garantisce una *forward compatibility* (compatibilità in avanti):

- Un programma compilato su un sistema vecchio gira su uno nuovo (Backward Compatibility);
- Un programma compilato su un sistema nuovo (il Docker con 18.04) NON gira su uno vecchio (MarssX86).

Poiché l'ambiente di compilazione (Host) dispone di librerie di sistema (`libc`) molto più recenti rispetto all'ambiente simulato (Guest OS nell'immagine disco), un linking dinamico causerebbe errori di runtime dovuti al version mismatch delle librerie condivise.

Una possibile soluzione è utilizzare anche in questo caso il linking statico, che disaccoppia il benchmark dalle librerie del sistema guest. Pertanto, l'unico modo per riuscire a far partire il compilatore MarssX86 è utilizzare l'opzione `-static` rendendo il programma indipendente dalla "vecchiaia" del sistema operativo simulato dentro MarssX86.

2.3 Strategia adottata

Per garantire un confronto il più equo possibile nonostante queste differenze strutturali, è stata adottata la seguente metodologia:

1. **Isolamento Microarchitetturale (Gem5):** Per le architetture ARM e RISC-V su Gem5, abbiamo scelto la modalità **SE**. Questa scelta ci permette di misurare le prestazioni pure

della CPU e della gerarchia di memoria (cache), eliminando il 'rumore' introdotto dai servizi del kernel e dallo scheduling dei processi;

2. **Realismo Operativo (MarssX86):** Per l'architettura x86 su MarssX86, operiamo in un contesto FS. Siamo consapevoli che i risultati includeranno l'overhead del sistema operativo (context switch, gestione TLB software);
3. **Mitigazione tramite Workload e Compilazione:**
 - Abbiamo scelto un workload CPU-bound (moltiplicazione matrici) e non I/O-bound. Questo minimizza le chiamate di sistema, rendendo le prestazioni dipendenti quasi esclusivamente dalla potenza di calcolo della CPU e dall'efficienza delle cache, riducendo il divario tra modalità SE e FS;
 - Abbiamo utilizzato la compilazione statica (-static) per tutti i binari. Questo garantisce che su Gem5 (SE) il simulatore possa emulare le syscall senza dipendenze esterne, e su MarssX86 (FS) evita conflitti di librerie (ABI mismatch) tra la macchina host e il sistema guest simulato.

2.4 Metodologia e Configurazione degli Esperimenti

Una volta stabilita la natura del workload e le modalità di simulazione, la fase sperimentale prevede l'esecuzione di una matrice di test strutturata per isolare l'impatto delle diverse configurazioni microarchitetturali.

2.4.1 Matrice dei test e definizione delle Baseline

Per ciascuna delle tre architetture analizzate (ARM, RISC-V e x86), verranno eseguite tre diverse configurazioni di simulazione, per un totale di 9 test. L'obiettivo è isolare l'impatto del modello di esecuzione della CPU e della gerarchia di memoria sulle prestazioni del workload.

Le varianti si basano su due modelli di processore fondamentali:

- **In-Order:** un modello in cui le istruzioni vengono eseguite nell'esatto ordine in cui appaiono nel programma. È un'architettura più semplice ed efficiente dal punto di vista energetico, ma soggetta a "stalli" (blocchi) se un'istruzione deve attendere dati dalla memoria.
- **Out-of-Order (OoO):** un modello avanzato che permette alla CPU di eseguire le istruzioni non appena le loro dipendenze sono soddisfatte, anche se non in ordine sequenziale. Questo permette di "nascondere" le latenze della memoria, ma richiede una logica hardware molto più complessa (scheduler, registri di rinomina, ecc.).

Per ogni architettura sono state definite le seguenti tre configurazioni:

1. **Configurazione 1 (Baseline):** Utilizza una CPU **Out-of-Order** ad alte prestazioni. La gerarchia di memoria prevede una cache L1 da 128 KB e una cache L2 da 2 MB. Rappresenta il target di massima potenza computazionale.
2. **Configurazione 2 (CPU Efficiency Test):** Mantiene le stesse cache della baseline (L1 128 KB, L2 2 MB) ma sostituisce il modello di core con una CPU **In-Order**. Questo test serve a misurare quanto l'architettura tragga beneficio dall'esecuzione fuori ordine per il particolare workload di moltiplicazione matriciale.
3. **Configurazione 3 (Memory Constraint Test):** Ritorna al modello di CPU **Out-of-Order** della baseline, ma riduce drasticamente le dimensioni delle cache (L1 a 32 KB e L2 a

256 KB). Questo permette di osservare quanto le prestazioni siano limitate dalla capacità della memoria (memory-bound) rispetto alla potenza pura di calcolo.

2.5 Esecuzione del workload per ARM e RISC-V (Gem5)

Per poter eseguire il workload scelto sulle architetture target (ARM e RISC-V), è prima necessario affrontare il problema della compatibilità binaria. Poiché le macchine utilizzate per lo sviluppo (Host) dispongono solitamente di un set di istruzioni x86_64 o ARM64 (Apple Silicon), i binari prodotti da un compilatore standard non sarebbero comprensibili dai simulatori configurati per architetture diverse.

In particolare, anche qualora si utilizzasse un processore Apple Silicon (già basato su ARM), sarebbe comunque necessaria la cross-compilazione (o più precisamente una compilazione specifica per il Guest) principalmente per due motivi:

- **ABI (Application Binary Interface):** il binario "nativo" di un Mac è progettato per macOS (formato Mach-O), mentre Gem5 richiede un binario in formato Linux ELF;
- **Librerie e System Call:** macOS utilizza librerie di sistema Apple, mentre il simulatore Gem5 emula l'interfaccia delle chiamate di sistema Linux.

2.5.1 Installazione dei cross-compilatori su Windows (x86_64)

Dunque, per prima cosa dobbiamo assicurarci di installare i "binari" giusti per la cross-compilazione. Nel sottosistema Ubuntu WSL, posizionati in `/home/NomeUtente/dev_local/gem5`, utilizzare i seguenti comandi, uno alla volta:

```
sudo apt-get update
```

Per l'architettura ARM:

```
sudo apt-get -y install gcc-aarch64-linux-gnu g++-aarch64-linux-gnu
```

Per verificare la corretta installazione del cross-compilatore lanciare il seguente comando:

```
aarch64-linux-gnu-gcc --version
```

Infine, per compilare il file C si utilizza il seguente comando (ricordando `-static`):

```
aarch64-linux-gnu-gcc -static matrix_mul.c -o matrix_mul_arm
```

Per l'architettura RISC-V:

```
sudo apt-get -y install gcc-riscv64-linux-gnu g++-riscv64-linux-gnu
```

Per verificare la corretta installazione del cross-compilatore lanciare il seguente comando:

```
riscv64-linux-gnu-gcc --version
```

Infine, per compilare il file C si utilizza il seguente comando (ricordando `-static`):

```
riscv64-linux-gnu-gcc -static matrix_mul.c -o matrix_mul_riscv
```

2.5.2 Installazione dei cross-compilatori su macOS (ARM)

I comandi rimangono identici a quelli utilizzati su x86_64 tranne che per qualche piccolo accorgimento. Nel capitolo 1.2.2 è stato utilizzato docker per l'installazione del simulatore. In questo caso, i comandi andranno utilizzati dalla seguente posizione /root/gem5:

```
root@{...}:/gem5#
```

Inoltre, gli stessi, devono essere utilizzati senza il comando "**sudo**" davanti poiché una volta entrati nell'ambiente docekr si è già un amministratore.

2.5.3 Comandi per la build del workload

Il comando per la build del workload (il file C) rimane invariato rispetto al calcolatore in cui viene utilizzato.

Ogni comando di build, all'interno di gem5/m5out/NomeCartella, creerà un file .txt che conterrà tutte le statistiche relative alla simulazione con le caratteristiche specificate.

Dunque, sia che ci si trovi in root/gem5 per macOS o in /home/NomeUtente/ dev_local/gem5 su un sistema Windows i comandi saranno i seguenti:

Le tre build per l'architettura ARM

```
// Comando di build per la configurazione 1 (baseline)
./build/ARM/gem5.opt --outdir=m5out/arm_baseline configs/deprecated/
example/se.py \
--cpu-type=ArmO3CPU \
--caches --l2cache \
--l1d_size=128kB --l1i_size=128kB \
--l2_size=2MB \
--sys-clock=2.688GHz \
--cmd=./matrix_mul_arm
```

```
// Comando di build per la configurazione 2 (CPU efficiency test)
./build/ARM/gem5.opt --outdir=m5out/arm_var1_cpu configs/deprecated/
example/se.py \
--cpu-type=ArmMinorCPU \
--caches --l2cache \
--l1d_size=12kB --l1i_size=128kB \
--l2_size=2MB \
--sys-clock=2.688GHz \
--cmd=./matrix_mul_arm
```

```
// Comando di build per la configurazione 3 (Memory constraint test)
./build/ARM/gem5.opt --outdir=m5out/arm_small_cache configs/deprecated/
example/se.py \
--cpu-type=ArmO3CPU \
--caches --l2cache \
--l1d_size=32kB --l1i_size=32kB \
--l2_size=256kB \
--sys-clock=2.688GHz \
--cmd=./matrix_mul_arm
```

Le tre build per l'architettura RISC-V

```
// Comando di build per la configurazione 1 (baseline)
./build/RISCV/gem5.opt --outdir=m5out/riscv_baseline configs/deprecated/
  example/se.py \
  --cpu-type=O3CPU \
  --caches --l2cache \
  --l1d_size=128kB --l1i_size=128kB \
  --l2_size=256kB \
  --sys-clock=2.688GHz \
  --cmd=./matrix_mul_riscv
```

```
// Comando di build per la configurazione 2 (CPU efficiency test)
./build/RISCV/gem5.opt --outdir=m5out/riscv_var1_cpu configs/deprecated/
  example/se.py \
  --cpu-type=MinorCPU \
  --caches --l2cache \
  --l1d_size=128kB --l1i_size=128kB \
  --l2_size=256kB \
  --sys-clock=2.688GHz \
  --cmd=./matrix_mul_riscv
```

```
// Comando di build per la configurazione 3 (Memory constraint test)
./build/RISCV/gem5.opt --outdir=m5out/riscv_small_cache configs/
  deprecated/example/se.py \
  --cpu-type=O3CPU \
  --caches --l2cache \
  --l1d_size=32kB --l1i_size=32kB \
  --l2_size=256kB \
  --sys-clock=2.688GHz \
  --cmd=./matrix_mul_riscv
```

Nota: cambiare O3CPU con ArmO3CPU ha senso per una questione di rigore metodologico e stabilità della simulazione per i seguenti motivi:

- **Specificità del modello:** Mentre O3CPU è un'etichetta generica, ArmO3CPU forza il simulatore a usare parametri di pipeline (come la profondità degli stadi e la dimensione del Reorder Buffer) specificamente modellati sulle caratteristiche delle CPU ARM;
- **Versioni future:** Il file se.py è considerato "deprecated". Nelle versioni più recenti di Gem5, l'automazione che associa O3CPU ad ArmO3CPU potrebbe essere rimossa, rendendo il comando generico non funzionante.

Questa cosa non vale in RISC-V dove il binario “build/RISCV/gem5.opt” contiene solo l'implementazione RISC-V quindi O3CPU non può essere altro che un processore RISC-V.

2.6 Esecuzione del workload per X86 (MarssX86)

A causa dell'architettura di MarssX86, che si basa su una versione datata di QEMU, l'interazione standard con il simulatore presenta significative limitazioni operative per l'esecuzione di benchmark automatizzati.

Inizialmente, la procedura standard prevedeva un intervento manuale dell'utente: era necessario avviare il workload nel sistema guest e, simultaneamente, inviare un comando al monitor di QEMU (tramite una combinazione di tasti rapida) per attivare la registrazione delle statistiche (simconfig -run). Questo approccio presentava due criticità fondamentali:

- **Mancanza di Determinismo:** Il tempo di reazione umano nell'impartire il comando di avvio non è costante. Questo rendeva impossibile sincronizzare perfettamente l'inizio della registrazione con l'inizio del calcolo matematico;
- **Inquinamento dei Dati ("Noise"):** Per dare all'utente il tempo fisico di attivare il simulatore, era necessario inserire nel codice C delle funzioni di ritardo (come sleep()) o loop a vuoto prima del calcolo. Tuttavia, il simulatore, essendo cycle-accurate, registrava anche questi cicli di attesa (fino a 160 milioni di cicli inutili nei test preliminari), falsando drasticamente metriche chiave come l'IPC (Instructions Per Cycle) e il conteggio totale delle istruzioni.

Per ovviare a queste problematiche e garantire la rigorosità scientifica dei dati, è stato necessario implementare una metodologia basata sulla **Region of Interest (ROI)**.

Questa tecnica sposta il controllo della simulazione dall'utente direttamente al codice sorgente. Tramite l'inclusione della libreria specifica ptlcalls.h e l'uso di "hypercalls" (istruzioni speciali che comunicano direttamente con l'hardware simulato), è il benchmark stesso a ordinare al simulatore di accendersi (ptlcall_switch_to_sim) esattamente un istante prima dell'inizio dell'algoritmo di moltiplicazione e di spegnersi (ptlcall_switch_to_native) appena terminato.

In questo modo, abbiamo eliminato completamente l'errore umano e il rumore di fondo, ottenendo misurazioni che riflettono esclusivamente il carico di lavoro computazionale oggetto di studio.

2.6.1 Implementazione della metodologia ROI

Dal momento in cui sta venendo utilizzato docker per entrambi i sistemi macOS e Windows, la procedura da eseguire rimane invariata. Possiamo suddividere la procedura in alcuni passi:

- **Passo 1: ottenimento della libreria ptlcall.h**

Subito dopo essere entrati nel container è necessario lanciare un comando per copiare il file ptlcalls.h all'interno della cartella "disks".

```
root@{...}:~/marss# cp /root/marss/marss/ptlsim/tools/ptlcalls.h /root/marss/disks/
```

- **Passo 2: modifica del codice C**

Il codice C rimarrà simile a quello utilizzato per l'esecuzione del workload con Gem5: verranno aggiunte solamente delle righe in più per poter aggiungere le librerie descritte prima. Per fare ciò, si utilizza il seguente comando:

```
root@{...}:~/marss# cd marss/disks
```

```
root@{...}:~/marss/marss/disks# cat > matrix_m.c <<EOF
#include <stdio.h>
#include <stdlib.h>
#include "ptlcalls.h" // La libreria per il controllo ROI

#define N 64
```



```

void matrix_multiply(double *A, double *B, double *C, int size) {
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            double sum = 0.0;
            for (int k = 0; k < size; k++) {
                sum += A[i * size + k] * B[k * size + j];
            }
            C[i * size + j] = sum;
        }
    }
}

int main() {
    // Usiamo static invece di malloc per evitare crash nell'ambiente
    // statico
    // La dimensione e' N*N * sizeof(double), gestita automaticamente
    static double A[N * N];
    static double B[N * N];
    static double C[N * N];

    printf("Inizializzazione dati (Veloce)...\n");

    // Inizializzazione deterministica
    for (int i = 0; i < N * N; i++) {
        A[i] = 1.0;
        B[i] = 2.0;
    }

    printf("--- ATTIVO LA SIMULAZIONE ORA ---\n");
    printf("(Il computer rallentera' drasticamente per simulare i
        calcoli floating point)\n");

    // === START ROI ===
    // Qui inizia la misurazione precisa
    ptlcall_switch_to_sim();

    matrix_multiply(A, B, C, N);

    // === STOP ROI ===
    // Qui finisce la misurazione
    ptlcall_switch_to_native();

    printf("--- SIMULAZIONE TERMINATA ---\n");
    printf("Benchmark Completato! Risultato cella 0: %f\n", C[0]);

    return 0;
}
EOF

```

- **Passo 3: compilazione del file C**

Per effettuare la compilazione di `matrix_mul.c`, eseguire i seguenti comandi uno alla volta:

```
root@{...}:~/marss/marss/disks# rm matrix_mul 2>/dev/null
```

Prima di installare il compilatore per i file C è necessario installare alcune librerie:

```
root@{...}:~/marss/marss/disks# apt-get update
```

```
root@{...}:~/marss/marss/disks# apt-get install musl-tools musl-dev
```

Non appena le librerie sono state installate, far partire il comando di compilazione:

```
root@{...}:~/marss/marss/disks# gcc -std=gnu99 -D_GNU_SOURCE -static -
nostartfiles -nostdinc \
    -I . \
    -I /usr/include/x86_64-linux-musl \
    -L /usr/lib/x86_64-linux-musl \
    /usr/lib/x86_64-linux-musl/crt1.o \
    /usr/lib/x86_64-linux-musl/crti.o \
    matrix_mul.c \
    /usr/lib/x86_64-linux-musl/crtn.o \
    -lc -o matrix_mul
```

Poiché l'ambiente virtualizzato (Guest) è isolato dal sistema ospitante (Host), è necessario creare un meccanismo di trasferimento per l'eseguibile compilato.

Quindi, tramite l'utility `genisoimage`, il file binario del benchmark (file C) viene incapsulato in un'immagine disco standard (ISO 9660). Questa immagine viene successivamente montata come unità CD-ROM virtuale all'avvio di QEMU, permettendo al sistema operativo Debian di leggere e copiare il file al suo interno.

```
root@{...}:~/marss/marss/disks# rm trasferimento.iso
```

```
root@{...}:~/marss/marss/disks# genisoimage -o trasferimento.iso
matrix_mul
```

- **Passo 4: configurazione delle CPU e delle Cache**

L'ultimo passaggio prima di procedere con l'esecuzione del workload è leggermente macchinoso. Infatti con il simulatore MarssX86 non è possibile specificare le tipologie di CPU o di Cache direttamente nel comando di build del workload (come si è fatto con Gem5) ma è necessario configurare alcuni file.

Scelta della CPU

Posizionati in `/root/marss/marss`, si andrà a creare una cartella "`my_configs`" all'interno della quale si inseriranno dei file che, oltre a specificare **il tipo di CPU** che si vuole utilizzare in base alla configurazione che si vuole buildare, presenta anche ulteriori comandi per **indicare alla distribuzione che file creare per salvare le informazioni importanti** relative al workload eseguito.

Proprio per questo motivo si rende necessario creare un ulteriore cartella dove salvare i file delle statistiche:

```
root@{...}:~/marss/marss# mkdir results
```

Ogni file presenta una configurazione di questo tipo:

```
-machine <tipo_di_cpu>
-logfile results/<nome_configurazione>.log
-stats results/cache_<nome_configurazione>.txt
-loglevel 2
```

All'interno di <nome_configurazione>.log vengono inserite le informazioni riguardanti cicli per istruzione (CPI), istruzioni per ciclo (IPC), tempo di simulazione, cicli di cpu e numero istruzioni, ecc. . . , invece, all'interno di cache_<nome_configurazione>.txt, vengono inserite, tra le altre, le informazioni riguardanti i Cache hit e i Cache miss di L1 e L2.

- **baseline.cfg**: viene utilizzato per la configurazione baseline. All'interno di esso, è specificata una CPU di tipo out-of-order, scrivendo (-machine single_core);

```
-machine single_core
-logfile results/baseline.log
-stats results/stats_baseline.txt
-loglevel 2
```

- **atom_core.cfg**: viene utilizzato per la configurazione CPU efficiency test. All'interno di esso è specificata una CPU in-order, scrivendo (-machine atom_core);

```
-machine atom_core
-logfile results/atom_core.log
-stats results/cache_atom.txt
-loglevel 2
```

- **small_cache.cfg**: viene utilizzato per la configurazione memory constraints test. All'interno di esso viene specificata la stessa CPU out-of-order della baseline poiché in questo caso ciò che cambia è la dimensione delle Cache.

```
-machine single_core
-logfile results/small_caches.log
-stats results/cache_smallC.txt
-loglevel 2
```

Note: se non viene resa possibile la creazione dei file tramite interfaccia grafica è necessario crearli direttamente da riga di comando utilizzando il comando "sudo".

Scelta delle Cache L1 ed L2

All'avvio di QEMU, le Cache L1 e L2 vengono preimpostate rispettivamente a 128KB e 2MB. Ciò significa che, dopo la build della prima e della seconda configurazione (entrambe con gli stessi valori di Cache L1 ed L2), per utilizzare delle Cache più piccole per la terza configurazione, sarà necessario ricompilare il simulatore prima dell'avvio di QEMU.

Per fare ciò vengono utilizzati i seguenti comandi (posizionati in root/marss/marss)

```
root@{...}:~/marss/marss# sed -i 's/SIZE: 128K/SIZE: 32K/g' config/
l1_cache.conf
```

```
root@{...}:~/marss/marss# sed -i 's/SIZE: 2M/SIZE: 256K/g' config/
l2_cache.conf
```

Successivamente è necessario pulire la vecchia compilazione per forzare la creazione del nuovo "binario" di compilazione, tramite il comando:

```
root@{...}:~/marss/marss# scons -c
```

Infine, si rieffettua la compilazione per apportare le modifiche sulle Cache:

```
root@{...}:~/marss/marss# scons -Q C=1 debug=0
```

2.6.2 Comandi per la build del workload

Dopo aver configurato i file per le CPU e capito come impostare le Cache, si può finalmente passare alla build del workload.

Configurazione 1: baseline

```
// Comando di build per la configurazione 1 (baseline)
root@{...}:~/marss/marss# ./qemu/qemu-system-x86_64 -m 1G \
-hda /root/marss/disks/debian_old.qcow2 \
-cdrom /root/marss/disks/trasferimento.iso \
-nographic \
-simconfig my_configs/baseline.cfg
```

Successivamente verranno chieste le credenziali per l'accesso all'interno di debian (username: root, password root) e si entrerà all'interno della distribuzione. Una volta dentro si monta il CD e si copia il file:

```
root@debian-amd64:~# mount /dev/cdrom /mnt
```

```
root@debian-amd64:~# cp /mnt/matrix_m .
```

Si danno i permessi di esecuzione, e poi si lancia:

```
root@debian-amd64:~# chmod +x matrix_m
```

```
root@debian-amd64:~# ./matrix_m
```

Arrivati a questo punto la build del workload è terminata e si può procedere con l'uscire dalla distribuzione e tornare al container tramite il seguente comando:

```
root@debian-amd64:~# poweroff
```

Le statistiche della build verranno salvate all'interno dei file specificati al "passo 4" in "Scelta della CPU" del capitolo 2.6.1.

Configurazione 2: CPU efficiency test

```
// Comando di build per la configurazione 2 (CPU efficiency test)
root@{...}:~/marss/marss# ./qemu/qemu-system-x86_64 -m 1G \
-hda /root/marss/disks/debian_old.qcow2 \
-cdrom /root/marss/disks/trasferimento.iso \
```

```
-nographic \  
-simconfig my_configs/atom_core.cfg
```

Dopo aver fatto partire la distribuzione con la CPU specificata, il procedimento da seguire per l'esecuzione del workload è il medesimo della **Configurazione baseline**.

Configurazione 3: Memory constraint test

In questo caso, dato che si vogliono utilizzare dei valori di Cache L1 e L2 differenti, prima di far partire la distribuzione con il comando sotto specificato, è necessario seguire il procedimento descritto al "**passo 4**" in "**Scelta delle Cache L1 e L2**" del capitolo 2.6.1.

```
// Comando di build per la configurazione 3 (Memory constraint test)  
root@{...}:~/marss/marss# ./qemu/qemu-system-x86_64 -m 1G \  
-hda /root/marss/disks/debian_old.qcow2 \  
-cdrom /root/marss/disks/trasferimento.iso \  
-nographic \  
-simconfig my_configs/small_cache.cfg
```

Dopo aver fatto partire la distribuzione con la CPU specificata, il procedimento da seguire per l'esecuzione del workload è il medesimo della **Configurazione baseline**.

3 Analisi dei dati raccolti

I dati raccolti non verranno analizzati solo in valore assoluto, ma verranno **normalizzati rispetto alla propria baseline**. Questo approccio permette di rispondere a domande del tipo: "Quale architettura beneficia maggiormente di un aumento della cache?" o "Quale ISA risulta più efficiente a parità di risorse?".

Il confronto finale avverrà dunque su due livelli:

1. **Intra-ISA:** Confronto tra le tre build della stessa architettura per valutarne la scalabilità.
2. **Inter-ISA:** Confronto tra i rapporti di performance delle tre architetture rispetto alle rispettive baseline, per determinare l'efficienza relativa dei set di istruzioni.

3.1 Analisi delle tre configurazioni per l'architettura ARM

L'analisi delle statistiche relative all'esecuzione del workload su gem5/ARM ha rivelato differenze significative nelle prestazioni e nell'efficienza del sistema, dovute sia alla microarchitettura della CPU, che alla gerarchia di memoria.

<i>Metrica</i>	<i>Baseline (Arm O3)</i>	<i>ArmMinorCPU</i>	<i>Small Caches (O3)</i>
Tempo di simulazione (s)	0,001186	0,005446	0,002132
Istruzioni Simulate	7.807.648	7.807.650	7.807.648
Cicli CPU	2.371.349	10.891.211	4.263.001
IPC (Instructions Per Cycle)	3,292	0,717	1,831
CPI (Cycles Per Instruction)	0,304	1,395	0,546
L1 D-Cache Hits	2.975.995	4.024.561	2.737.002
L1 D-Cache Misses	8943	1878	264.751
L1 D-Cache Miss Rate	0,30%	0,05%	8,82%
L1 I-Cache Hits	572.369	1.113.779	589.353
L1 I-Cache Misses	462	413	468
L1 I-Cache Misses Rate	0,08%	0,04%	0,08%
L2 Cache Hits	60	80	254.708
L2 Cache Misses	2096	2134	2096
L2 Cache Misses Rate	97,22%	96,39%	0,82%
Branch Cond. Incorrect	4637	4715	4636
Branch Mispredicts (Commit)	4548	N/D*	4548

Table 1: Dati di analisi per ARM

3.1.1 Statistiche generali su CPU e istruzioni

La differenza prestazionale più marcata risiede nella scelta della microarchitettura del core. Il modello **ArmO3** (Out-of-Order) risulta circa 4,6 **volte più veloce** rispetto al modello **Minor** (In-Order). Questa disparità è chiaramente riflessa nel valore dell'IPC (Instructions Per Cycle), che crolla da un ottimo 3,29 nella configurazione baseline a un modesto 0,72 nel caso del core Minor.

Il core O3 riesce a superare la barriera delle 3 istruzioni per ciclo sfruttando l'esecuzione fuori ordine e il parallelismo a livello di istruzione (ILP). Al contrario, il core Minor fatica a

completare più di una singola istruzione per ciclo a causa della sua natura *in-order* e di risorse di *issue* limitate; ciò si traduce in un tempo di esecuzione totale significativamente più lungo a parità di carico di lavoro. Nella configurazione con **cache ridotte**, nonostante la CPU rimanga la stessa (O3), si osserva un degrado dell'IPC che scende a 1,83. Questo dimezzamento delle prestazioni è dovuto ai frequenti stalli della pipeline in attesa dei dati recuperati dalla cache L2, la cui latenza è superiore a quella della L1.

3.1.2 Statistiche Cache

Cache hit e Cache Miss L1

L'utilizzo di cache ridotte trasforma la memoria in un collo di bottiglia: il *miss rate* della L1 Data Cache balza dallo 0,3% all'8,8%, con un incremento dei miss di circa 30 volte. Curiosamente, la Minor CPU registra un numero di *hit* totali molto più alto rispetto alla O3. Questo non dipende dal numero di istruzioni (che resta identico), ma dal fatto che il modello *in-order* può generare accessi ripetuti o gestire il *front-end* con *fetch* più frequenti e meno aggregati rispetto alla struttura complessa della O3.

La **I-Cache** (Instruction Cache) rimane invece estremamente stabile e con miss bassissimi in tutte le configurazioni, suggerendo che il codice del workload sia molto compatto e risieda agevolmente anche in spazi ridotti.

Cache hit e Cache Miss L2

L'inefficienza della L1 nelle configurazioni con cache ridotte sposta il carico sulla L2, dove gli *hit* passano da appena 60 a oltre 254.000. Il numero di *L2 Misses* assoluti, tuttavia, resta pressoché identico in tutti i casi (circa 2100). Questo indica che il *working set* totale del programma entra perfettamente nella L2 e che i miss registrati sono esclusivamente **Compulsory Misses** (dati caricati dalla DRAM solo all'inizio).

Si osserva un paradosso statistico nel *miss rate* della L2: esso appare altissimo nella baseline (97,22%) e bassissimo nel caso *small cache* (0,82%). Questo accade perché nella baseline la L1 filtra quasi tutto, mandando in L2 solo le richieste "impossibili" (che falliscono); nel caso delle cache ridotte, la L1 manda in L2 una quantità enorme di richieste "facili" che hanno successo, diluendo il numero di miss e abbassando artificialmente la percentuale, nonostante il sistema stia operando più lentamente a causa della latenza superiore della L2.

3.1.3 Analisi dei Branch

Le statistiche sulla predizione dei rami mostrano valori simili tra le configurazioni O3 (4548 mispredizioni), confermando l'identità del percorso di esecuzione del carico di lavoro. Il modello Minor CPU presenta un numero leggermente superiore di previsioni errate (4715), probabilmente a causa di un hardware di predizione meno sofisticato.

È fondamentale distinguere tra le metriche riportate:

- **Cond. Incorrect:** indica quante volte il predittore ha sbagliato la direzione di un salto condizionale (derivante da istruzioni come *if* o cicli).
- **Branch Mispredicts (Commit):** è un dato globale che include tutti i tipi di salti (ritorni di funzione, salti indiretti, ecc.) che hanno completato l'intera pipeline.

Nella tabella, la voce *Commit* per **ArmMinorCPU** è contrassegnata come **N/D*** per ragioni architettoniche. Nelle CPU O3, il *Commit* (o *Retire*) è l'ultima fase della pipeline, dove i

risultati diventano permanenti e il processore è certo che l'istruzione dovesse essere eseguita. Le CPU O3 usano la **speculazione**: scommettono sulla direzione di un salto e lavorano preventivamente. Se la scommessa è errata, il lavoro viene "buttato via" (*squash*). La statistica di *Commit* conta solo gli errori che arrivano alla fine di questa catena, rappresentando l'impatto reale sulle prestazioni.

Nella CPU Minor, essendo *in-order*, non esiste uno stadio di *Commit* separato: le istruzioni fluiscono linearmente e, se un salto è errato, la pipeline viene resettata immediatamente nello stadio di *Execute*. Pertanto, gem5 utilizza il parametro *condIncorrect* per la Minor, mentre per la O3 il dato di *Commit* risulta più preciso per misurare il rallentamento effettivo. Il "costo" di ogni errore di predizione aumenta drasticamente con cache piccole: se il dato per risolvere il salto è in L2 anziché in L1, la CPU O3 specula "al buio" per più tempo lungo il percorso errato, caricando istruzioni inutili e sprecando cicli.

3.1.4 Conclusioni

L'analisi comparativa tra le diverse configurazioni architetturali e gerarchie di memoria permette di trarre conclusioni fondamentali sul bilanciamento di un sistema di calcolo:

- **Dominanza dell'architettura Out-of-Order:** per il carico di lavoro analizzato, il core O3 si è dimostrato superiore in termini di throughput. Il parallelismo a livello di istruzione (ILP) estratto dal core compensa ampiamente la complessità hardware, permettendo di completare il task in una frazione del tempo richiesto dal core Minor;
- **Criticità della Cache L1:** la dimensione della cache di primo livello è il vero ago della bilancia prestazionale. Il workload presenta un *working set* che risiede agevolmente nella L1 standard, ma l'impiego di una cache ridotta causa un effetto a catena: l'incremento del miss rate sposta il carico sulla L2, introducendo latenze che nemmeno un core avanzato come l'O3 riesce a mascherare, portando a un rallentamento medio del 45% (fino a rendere l'O3 meno efficiente di un core In-Order in ambito RISC-V);
- **Limiti del core In-Order:** il modello Minor è strutturalmente limitato dal proprio front-end. La sua natura *single-issue* non permette di sfruttare appieno le unità di esecuzione anche quando i dati sono disponibili in cache, rendendolo una scelta valida solo in contesti di estremo risparmio energetico o dove le risorse di memoria sono talmente scarse da annullare i benefici della speculazione;
- **Saturazione della Gerarchia:** i circa 2100 miss costanti in L2 (per ARM) e 3010 (per RISC-V) rappresentano i **Compulsory Misses** del programma. Questo dato indica che il limite fisico minimo di accessi alla memoria principale (DRAM) è intrinseco al codice stesso e rimane invariato indipendentemente dalla potenza del processore o dalla dimensione delle cache intermedie.

In conclusione, l'esperimento evidenzia che una CPU potente è efficace solo se supportata da un sistema di memoria equilibrato. La scelta dell'architettura deve quindi basarsi non solo sulla velocità teorica del core, ma sulla capacità della gerarchia di memoria di alimentare costantemente la pipeline, evitando che la speculazione diventi un costo anziché un vantaggio.

3.2 Analisi delle tre configurazioni per l'architettura RISC-V

<i>Metrica</i>	<i>Baseline (RISCV O3)</i>	<i>Minor_RISCV</i>	<i>Small Caches (O3)</i>
Tempo di simulazione (s)	0,003567	0,005997	0,006364
Istruzioni Simulate	10.043.070	10.043.070	10.043.070
Cicli CPU	7.134.755	11.994.951	12.728.034
IPC (Instructions Per Cycle)	1,408	0,837	0,789
CPI (Cycles Per Instruction)	0,710	1,194	1,267
L1 D-Cache Hits	3.551.546	4.050.839	3.315.652
L1 D-Cache Misses	16.645	3681	252.600
L1 D-Cache Miss Rate	0,47%	0,09%	7,08%
L1 I-Cache Hits	600.802	1.150.882	600.828
L1 I-Cache Misses	619	537	629
L1 I-Cache Misses Rate	0,10%	0,05%	0,10%
L2 Cache Hits	40	40	235.932
L2 Cache Misses	3010	3056	3010
L2 Cache Misses Rate	98,69%	98,71%	1,26%
Branch Cond. Incorrect	4816	4898	4817
Branch Mispredicts (Commit)	4725	N/D*	4729

Table 2: Dati di analisi per RISC-V

3.2.1 Statistiche generali su CPU e istruzioni

Nell'architettura RISC-V, la CPU **O3** (Out-of-Order) si dimostra circa 1,7 **volte più veloce** della **Minor CPU** in termini di tempo di simulazione nella configurazione baseline. L'IPC della baseline risulta pari a 1,408, un valore che evidenzia la capacità del core di processare più di un'istruzione per ciclo grazie all'esecuzione fuori ordine e al parallelismo a livello di istruzione (ILP). Al contrario, il core Minor, vincolato dall'esecuzione sequenziale (*in-order*), subisce maggiormente l'impatto di stalli strutturali e dipendenze tra i dati, portando il tempo di esecuzione a quasi 6 ms contro i 3,5ms della baseline.

Tuttavia, l'efficienza del core O3 è strettamente legata alla gerarchia di memoria. Nella configurazione con **cache ridotte**, la CPU O3 subisce un degrado tale da diventare la più lenta delle tre configurazioni (6,36 ms), risultando persino meno efficiente del modello Minor. Questo accade perché l'architettura fuori ordine è estremamente sensibile alla latenza: quando la L1 D-Cache diventa un collo di bottiglia (con un *miss rate* che balza dallo 0,47% al 7,08%), le strutture interne della CPU si saturano in attesa dei dati. In questo scenario, l'IPC crolla a 0,789 e la complessità del core O3 diventa un onere, poiché il processore occupa più tempo a gestire stalli e speculazioni fallite che a eseguire codice utile.

3.2.2 Statistiche Cache

Cache hit e Cache Miss L1

L'analisi dei flussi rivela una **I-Cache** (Instruction Cache) estremamente stabile in tutte le configurazioni (miss rate 0,10%), indicando che il codice eseguibile è compatto e risiede agevolmente in memoria.

La criticità è invece concentrata sulla **L1 D-Cache**: nella baseline essa filtra la quasi totalità delle richieste, ma nella versione *small cache* subisce un'espulsione massiccia di dati, portando il numero di miss da circa 16k a oltre 252k.

Cache hit e Cache Miss L2

Nella configurazione baseline, la L2 riceve solo circa 3000 richieste, che falliscono quasi integralmente (*miss rate* 98,7%). Si tratta di **Compulsory Misses**, ovvero il caricamento iniziale di dati mai visti prima. Nella versione con cache ridotte, la L2 viene invece investita da oltre 235.000 accessi (hits) precedentemente gestiti dalla L1. Sebbene il *miss rate* della L2 scenda paradossalmente all'1,26%, ciò riflette solo il fatto che la L2 sta supplendo alle carenze della L1, introducendo però una latenza d'accesso superiore che penalizza l'intero sistema.

3.2.3 Analisi dei Branch

Il numero di predizioni errate è molto simile tra le due versioni O3 (4729 mispredicts al commit), confermando che il percorso logico del software non cambia. Nella configurazione con cache ridotte, pur restando costante il numero di errori, la CPU paga un prezzo molto più alto in termini di cicli di stallo, poiché i miss in L1 ritardano il recupero dei dati necessari a risolvere la condizione del salto.

Confronto O3 vs Minor e impatto della latenza

Analizzando la distinzione tra *condIncorrect* (errori sui salti condizionali) e *commit.branchMispredicts* (dato globale al ritiro), si osserva che:

- **Configurazione Minor**: come previsto, la voce *Commit* riporta N/D* per l'assenza di uno stadio di ritiro separato. Il valore di *condIncorrect* (4898) è leggermente superiore ai modelli O3, suggerendo che il predittore integrato nel core Minor sia meno raffinato.
- **Latenza di risoluzione**: nella versione con cache ridotte, l'alto tasso di miss in L1 D-Cache rallenta drasticamente il calcolo delle condizioni nello stadio di *Execute*. La CPU rimane in uno stato di "incertezza speculativa" per un tempo prolungato, causando un incremento delle istruzioni *squashed* (caricate inutilmente lungo percorsi errati) che contribuisce al crollo dell'IPC.

3.2.4 Conclusioni

L'analisi comparativa su architettura RISC-V mette in luce dinamiche differenti rispetto a quanto osservato per ARM, evidenziando un equilibrio ancora più precario tra potenza di calcolo e sottosistema di memoria:

- **Superiorità condizionata dell'architettura Out-of-Order**: nella configurazione baseline, il core **O3** si dimostra circa 1,7 volte più veloce del modello Minor. Tuttavia, l'IPC di 1,408 suggerisce che il parallelismo estratto (ILP) sia inferiore rispetto ad altre architetture, rendendo il vantaggio della speculazione meno netto e più dipendente dall'efficienza delle cache.
- **Il "sorpasso" della Minor CPU**: a differenza di ARM, in RISC-V la criticità della cache L1 è tale da ribaltare le gerarchie di potenza. Nella configurazione **Small Caches**, il core O3 diventa il meno efficiente del gruppo (6,36ms), venendo superato in termini di tempo di esecuzione persino dal core **Minor** (5,99ms). Questo dimostra che, se la memoria non è in grado di alimentare la pipeline, la complessità dell'esecuzione fuori ordine diventa un

ostacolo anziché un beneficio.

- **Prevedibilità del core In-Order:** il modello Minor, pur essendo limitato dalla sua natura *single-issue* e dalla mancanza di uno stadio di commit complesso, si dimostra più resiliente alle variazioni del sistema di memoria. In scenari con risorse limitate, la sua esecuzione lineare evita i pesanti costi di "svuotamento" della pipeline (squash) che affliggono il core O3 quando la latenza di risoluzione dei branch aumenta.
- **Saturazione della Gerarchia:** i circa 3010 miss costanti registrati in L2 rappresentano i **Compulsory Misses** del carico di lavoro RISC-V. Questo valore indica la soglia minima di accessi alla DRAM necessaria per caricare il set di dati iniziale; il fatto che tale numero resti invariato conferma che il collo di bottiglia prestazionale risiede esclusivamente nella latenza di accesso ai dati (L1 vs L2) e non nel volume totale dei dati stessi.

In conclusione, l'esperimento su RISC-V sottolinea un principio fondamentale della progettazione hardware: **un core complesso richiede una gerarchia di memoria proporzionata**. Senza una cache L1 adeguata, l'architettura O3 "affama" se stessa, rendendo paradossalmente preferibile una microarchitettura più semplice e lineare come quella del core Minor.

3.3 Analisi delle tre configurazioni per l'architettura MarssX86

L'analisi condotta sull'architettura x86 tramite il simulatore Marss ha l'obiettivo di isolare il comportamento del workload (User) rispetto all'overhead introdotto dal sistema (Total) al variare delle configurazioni microarchitetture, poiché MarssX86 è un emulatore Full System (FS).

<i>Metrica</i>	<i>Baseline (OoO)</i>	<i>Var 1: Atom</i>	<i>Var 2: Small Caches</i>
Tempo Host (s)	9	6	9
Tempo simulato (ms)	4,6409	8,4707	4,8709
Cicli CPU	12.474.800	22.769.162	13.092.853
Istruzioni	7.760.158	7.448.858	7.981.476
IPC	0,6221	0,3271	0,6096
CPI	1,6075	3,0567	1,6404
L1i Hit	1.191	1.641	4.490
L1i Miss	519	482	3.236
L1i Miss Rate	30,35%	22,70%	41,88%
L1d Hit	3.273.198	4.253.740	3.292.506
L1d Miss	1.869	1.847	43.010
L1d Miss Rate	0,06%	0,04%	1,29%
L2 Hit	56	71	41.712
L2 Miss	2.332	2.258	4.534
L2 Miss Rate	97,65%	96,95%	9,80%
Branch Mispred.	4.561	5.020	6.567
Branch Cond. Inc.	4.539	5.020	6.408

Table 3: Confronto metriche generali di simulazione (Total)

<i>Metrica</i>	<i>Baseline (OoO)</i>	<i>Var 1: Atom</i>	<i>Var 2: Small Caches</i>
Tempo simulato (ms)	4,6123	8,4273	4,6774
Cicli CPU	12.397.779	22.652.712	12.572.917
Istruzioni	7.733.613	7.430.872	7.733.133
IPC	0,6238	0,3280	0,6151
CPI	1,6031	3,0485	1,6259
L1i Hit	1.191	61	4.490
L1i Miss	519	20	3.236
L1i Miss Rate	30,35%	24,69%	41,88%
L1d Hit	3.265.122	4.247.880	3.226.082
L1d Miss	1.139	1.201	40.130
L1d Miss Rate	0,03%	0,03%	1,23%
L2 Hit	21	68	40.368
L2 Miss	1.637	1.153	2.998
L2 Miss Rate	98,73%	94,43%	6,91%
Branch Mispredict (commit)	4.199	4.182	4.176
Branch Cond. Inc.	4.194	4.182	4.172

Table 4: Tabella USER (Solo Workload)

Note: Questa tabella isola l'efficienza pura dell'algoritmo di moltiplicazione matrici.

3.3.1 Statistiche generali su CPU e istruzioni

Il primo confronto riguarda l'efficienza di esecuzione delle tre varianti: Baseline (Out-of-Order), Atom (In-Order) e Small Caches (Out-of-Order con cache ridotta).

Osservando i dati User, che rappresentano il puro carico computazionale della moltiplicazione di matrici, notiamo che il numero di istruzioni rimane costante (~ 7.73 milioni), confermando la correttezza deterministica della simulazione. Tuttavia, i Cicli CPU variano drasticamente:

- La **configurazione Atom** richiede il 82% di cicli in più (22.6M) rispetto alla Baseline (12.4M) per eseguire lo stesso codice. Questo crollo dell'IPC (da 0,62 a 0,33) è intrinseco alla natura In-Order della pipeline, che non può mascherare le latenze di esecuzione tramite l'esecuzione fuori ordine.
- La **configurazione Small Caches**, nonostante la drastica riduzione della memoria, mostra una resilienza notevole: l'IPC User cala solo marginalmente (0,615 vs 0,624). Questo dimostra che il motore Out-of-Order riesce a nascondere efficacemente le latenze aggiuntive introdotte dal sottosistema di memoria.

Un dato interessante riguarda il Tempo di Simulazione (Host): la configurazione Atom completa la simulazione in meno tempo (6s) rispetto alle versioni OoO (9s). Ciò è dovuto alla minore complessità del modello matematico della CPU In-Order, che non deve simulare strutture complesse come il Reorder Buffer o la rinomina dei registri.

3.3.2 Statistiche Cache

L'analisi della gerarchia di memoria evidenzia come la dimensione delle cache influenzi il traffico dati tra la CPU e la memoria principale.

Cache hit e Cache Miss L1

La Cache L1 Dati (L1d) si rivela il componente critico. Nella Baseline e nella configurazione Atom (128kB), i miss User sono trascurabili (~ 1.100), indicando che l'intero working set delle matrici risiede comodamente nella cache. Nella configurazione Small Caches (32kB), invece, assistiamo a un fenomeno di *capacity miss*: i miss User salgono a 40.130.

La cache è troppo piccola per contenere i blocchi di matrice riutilizzati dall'algoritmo, costringendo la CPU a continue espulsioni e ricaricamenti.

È interessante notare come il core Atom generi circa 1 milione di accessi L1 in più rispetto alla Baseline: mancando di meccanismi di *Store-to-Load Forwarding*, la CPU In-Order è costretta a formalizzare ogni passaggio di dati attraverso la cache.

Cache hit e Cache Miss L2

La Cache L2 agisce come "rete di salvataggio".

- Nelle **configurazioni con L1 grande (Baseline/Atom)**, la L2 è scarsamente utilizzata (pochi accessi, quasi tutti cold miss), risultando in un miss rate elevato ma irrilevante in termini assoluti.
- Nella **configurazione Small Caches**, la L2 diventa fondamentale. Essa intercetta quasi la totalità dei dati espulsi dalla L1: su 43.000 accessi totali, ben 41.712 sono Hit.

Questo spiega perché le prestazioni della Small Caches non crollano: sebbene la L1 fallisca spesso, i dati vengono recuperati dalla L2 (molto più veloce della RAM), mitigando la penalità temporale.

3.3.3 Analisi dei Branch

L'analisi della predizione dei salti rivela un'interessante correlazione indiretta con la memoria.

Confrontando i dati Total (Sistema completo), si nota un aumento dei *Branch Mispredictions* nella configurazione Small Caches (6.567) rispetto alla Baseline (4.561). Tuttavia, isolando i dati User, il numero di errori rimane pressoché identico (~ 4.180) per tutte le configurazioni.

Questa discrepanza porta a una conclusione importante: Il peggioramento apparente nella predizione dei salti non è dovuto a una difficoltà del processore nel prevedere il flusso del programma di moltiplicazione (che rimane regolare), ma è un effetto collaterale dello stress sulla memoria. L'alto numero di miss in L1 nella versione Small Caches costringe il sistema operativo a intervenire più spesso (gestione page fault, buffer, interrupt). Il codice del Kernel, essendo più irregolare, induce il predittore in errore più frequentemente. Pertanto, il calo di efficienza del front-end è una conseguenza diretta del collo di bottiglia nel back-end (memoria).

3.3.4 Conclusioni

L'analisi condotta sull'architettura MarssX86 permette di trarre tre conclusioni fondamentali riguardo l'interazione tra microarchitettura, memoria e sistema operativo.

In primo luogo, si conferma la **superiorità dell'esecuzione Out-of-Order** per carichi di lavoro *memory-intensive* come la moltiplicazione di matrici. La configurazione Baseline (OoO) ha completato il task con la metà dei cicli rispetto alla configurazione Atom (In-Order), dimostrando come la complessità hardware (rinomina dei registri, esecuzione speculativa)

sia necessaria per mascherare le latenze di accesso ai dati, mantenendo un CPI vicino a 1,6 contro il 3,0 dell'Atom.

In secondo luogo, l'esperimento ha evidenziato la **resilienza della pipeline OoO** di fronte a risorse scarse. Nonostante la riduzione della Cache L1 abbia causato un aumento dei miss di oltre il 2000% (da ~ 1.800 a ~ 43.000), l'IPC del workload utente è rimasto quasi invariato (0,615 vs 0,624).

Questo risultato controintuitivo dimostra che un moderno processore OoO, supportato da una Cache L2 capiente ed efficiente, è in grado di tollerare fallimenti massicci nel primo livello di memoria senza stallare l'esecuzione del programma principale.

Infine, la disaggregazione dei dati tra User e Kernel ha svelato il vero costo del sottodimensionamento della cache: l'**overhead di sistema**. Il degrado delle prestazioni globali nella configurazione *Small Caches* non è imputabile al rallentamento dell'algoritmo di calcolo, bensì all'esplosione dell'attività del Sistema Operativo (+830% di istruzioni kernel), costretto a intervenire massicciamente per gestire la congestione del sottosistema di memoria.

Ne consegue che, in scenari di progettazione embedded o low-power, il dimensionamento della cache L1 non deve essere valutato solo in funzione del throughput dell'applicazione, ma anche per garantire la stabilità e l'efficienza dei servizi di sistema.

3.4 Analisi delle baseline tra le architetture

<i>Metrica</i>	<i>Baseline (Arm O3)</i>	<i>Baseline (RISC-V)</i>	<i>Baseline user (MarssX86)</i>
Tempo di simulazione	0,001186s	0,003567s	4,6123ms
Istruzioni Simulate	7.807.648	10.043.070	7.449.759
Cicli CPU	2.371.349	7.134.755	12.474.800
IPC (Instructions Per Cycle)	3,292	1,408	0,597
CPI (Cycles Per Instruction)	0,304	0,710	1,675
L1 D-Cache Hits	2.975.995	3.551.546	3.265.122
L1 D-Cache Misses	8.943	16.645	1.139
L1 D-Cache Miss Rate	0,30%	0,47%	0,03%
L1 I-Cache Hits	572.369	600.802	1.191
L1 I-Cache Misses	462	619	519
L1 I-Cache Misses Rate	0,08%	0,10%	30,35%
L2 Cache Hits	60	40	21
L2 Cache Misses	2096	3010	1.637
L2 Cache Misses Rate	97,22%	98,69%	97,6%
Branch Cond. Incorrect	4637	4816	4.199
Branch Mispredicts (Commit)	4548	4725	4.561

Table 5: Tabella delle tre baseline a confronto

Note: Per MarssX86 sono stati isolati i dati relativi al solo workload utente.

3.4.1 Statistiche generali su CPU e istruzioni

L'analisi delle prestazioni generali illustrata in tabella 5, rivela un netto vantaggio per l'architettura ARM O3, che svetta con un IPC di 3,292. Questo indica una pipeline estremamente efficiente nel gestire il parallelismo a livello di istruzione (ILP) per il workload di moltiplicazione di matrici, completando il compito in soli 1,18 ms simulati.

Il RISC-V si posiziona in una fascia intermedia con un IPC di 1,408. Sebbene esegua un numero maggiore di istruzioni totali (10,04M) rispetto ad ARM, la sua efficienza per ciclo è inferiore, portando a un tempo di simulazione tre volte superiore (3,56 ms).

La baseline di MarssX86 mostra invece l'IPC più basso (0,597). È fondamentale sottolineare che, avendo isolato la componente *user* (il workload applicativo) dalla componente *kernel* (il sistema operativo), l'IPC registrato riflette la complessità della decodifica x86 e la presenza di stalli nella pipeline che persistono nonostante l'ottimizzazione del codice utente.

3.4.2 Statistiche Cache

L'efficienza del sottosistema di memoria gioca un ruolo fondamentale nel determinare i risultati prestazionali sopra descritti.

Cache hit e Cache Miss L1

Isolando le statistiche del workload (*User mode*) in MarssX86, emerge un dato estremamente positivo per la L1 D-Cache: il miss rate scende allo 0,03%, il valore più basso tra le tre architetture. Questo dimostra che l'algoritmo di moltiplicazione di matrici è perfettamente ottimizzato per la gerarchia di memoria x86.

Tuttavia, persiste un miss rate anomalo nella L1 I-Cache (30,35%). Analizzando il numero assoluto di accessi (1.710), si deduce che tale valore è un artefatto statistico: trattandosi di un workload breve, i *cold miss* (miss obbligatori al primo caricamento delle istruzioni) pesano sproporzionatamente sulla statistica finale, a differenza di ARM e RISC-V dove il volume di istruzioni simulate permette di ammortizzare questi costi iniziali.

Cache hit e Cache Miss L2

Per tutte e tre le architetture, il miss rate in L2 è prossimo al 98%. Questo fenomeno è tipico di workload i cui dati risiedono quasi interamente in L1 (128 kB). Poiché la L1 filtra con successo quasi tutti gli accessi del workload utente (come confermato dal miss rate dello 0,03% su x86), alla L2 giungono pochissime richieste, rendendo statisticamente dominante ogni singolo miss dovuto a dati non ancora mappati.

3.4.3 Analisi dei Branch

Nonostante le differenze di throughput, il comportamento del *Branch Predictor* è coerente tra le architetture, con errori di predizione che oscillano tra 4.500 e 4.800.

Come evidenziato dai dati, per i modelli Out-of-Order (O3) il parametro di riferimento è il *Branch Mispredicts (Commit)*. In queste CPU, il processore esegue istruzioni in modo speculativo: se la "scommessa" sul salto è errata, l'errore viene rilevato e contabilizzato solo nella fase finale di **Commit**. I circa 4.500 errori rappresentano i salti che hanno effettivamente causato il *flush* (svuotamento) della pipeline, impattando direttamente sul CPI finale.

In sintesi, l'isolamento della componente *user* in MarssX86 conferma che il collo di bottiglia non risiede nella gestione dei dati (D-Cache estremamente efficiente), ma nella penalità di decodifica x86 e nei miss iniziali di caricamento delle istruzioni, che in un workload così breve condizionano pesantemente l'IPC finale.

3.4.4 Conclusioni

In conclusione, il confronto tra le tre baseline evidenzia come l'architettura **ARM O3** sia la più performante per il workload di calcolo matriciale, grazie a un bilanciamento ottimale tra velocità di clock simulata e gestione della pipeline. Il **RISC-V** si conferma un'architettura solida con un'efficienza predicibile, mentre **MarssX86** rivela che, nonostante un'eccellente gestione dei dati (D-Cache), soffre di un overhead maggiore nella gestione del flusso di istruzioni. L'analisi "User-only" è stata fondamentale per distinguere le reali prestazioni dell'algoritmo dal rumore di fondo generato dal sistema operativo simulato, permettendo un confronto più equo tra le diverse tecnologie.